Lecture L21

*Blind review — please score this lecture on row **L21** in the Excel form (1–10 per dimension).*

Backpropagation in Neural Networks

Learning Objectives

By the end of this lecture, you will be able to:

1. **Remember** the two-phase structure of training: the forward pass computes and stores intermediate values; the backward pass computes gradients.
2. **Understand** why naive gradient computation is exponentially expensive and how dynamic programming eliminates this redundancy via reverse topological ordering.
3. **Apply** the backpropagation algorithm by hand to a two-layer neural network, computing every partial derivative step-by-step for a given minibatch.
4. **Analyze** a computational graph to identify which intermediate results must be stored during the forward pass and why each is necessary for the backward pass.
5. **Create** the backward pass for a novel operation (e.g., batch normalization) by deriving its local Jacobian and integrating it into the gradient flow.

---

1. The Problem That Backpropagation Solves

Imagine you've just trained a neural network on a massive dataset. The training loop reports a loss, but now you need to update one million parameters. Each parameter demands its own partial derivative of the cost function J. If you compute each derivative independently — tracing every path from that parameter to the output — you'll be waiting until the heat death of the universe.

Why? Because the number of computational paths from a parameter to the output grows exponentially with network depth. A network with n nodes per layer and L layers has on the order of n^{L-1} distinct paths. Computing each path's product independently means you repeat the same sub-chain products millions of times. This is the computational equivalent of recalculating 3×4 every time you need it, instead of remembering the answer.

The central tension: We need one million derivatives, each a sum over potentially millions of paths. Yet training happens in minutes, not millennia. Something must give.

Backpropagation resolves this tension by computing all one million derivatives in roughly the same time as *two* forward passes. It achieves this by recognizing that if two parameters share a subexpression, that subexpression's derivative is computed once and shared. This is dynamic programming applied to gradient computation — and it's the same algorithmic insight that makes sequence alignment algorithms in bioinformatics efficient. Just as the Smith-Waterman algorithm reuses subproblem scores to avoid re-enumerating all possible alignments, backpropagation reuses intermediate gradients to avoid re-enumerating all possible paths.

Try this yourself: Before reading further, estimate how many distinct paths exist from an input to the output in a fully-connected network with 3 hidden layers of 100 neurons each. Now imagine computing a derivative for each path independently.

Answer Each layer has 100 neurons, and each neuron in layer k connects to all 100 neurons in layer $k+1$. A path from an input neuron to the output neuron must choose one neuron in each hidden layer:

$100 \times 100 \times 100 = 10^6$ paths for a single parameter. With one million parameters, naive computation would trace 10^{12} path-products, each repeating the same intermediate computations millions of times. Backpropagation collapses this to roughly 2×10^6 operations.

---

2. The Computational Graph

Before deriving the algorithm, we need a way to visualize the flow of computation. Any differentiable function can be represented as a **directed acyclic graph (DAG)**:

- **Nodes** represent either variables (data, parameters, intermediate results) or operations (addition, multiplication, activation functions).
- **Edges** point from inputs to the operation that consumes them. An edge $u^{(i)} \to u^{(j)}$ means $u^{(i)}$ is an input to the operation computing $u^{(j)}$.

Concrete example: The expression $z = x \cdot y + b$ becomes:

```
x --→ [×] --→ v --→ [+] --→ z
y --→      b --→
```

Here $v = x \cdot y$ is an intermediate variable (node). The operation nodes are $[\times]$ and $[+]$.

Why this matters: The graph makes dependencies explicit. During backpropagation, we traverse this graph in reverse order. Every edge becomes a conduit for a gradient signal. Think of it as a plumbing system: during the forward pass, water flows from inputs to output. During the backward pass, we inject dye at the output and trace it backward through every pipe, measuring how much each upstream valve contributed to the final color.

Key terminology: - **Leaf nodes:** Nodes with no incoming edges (parameters, inputs). These are what we differentiate *with respect to*. - **Root node:** The final scalar output (the cost J). This is where gradient computation *starts*. - **Upstream gradient:** The gradient arriving at a node's output, already computed from nodes closer to the root. - **Downstream gradient:** The gradient we send to that node's inputs, which we compute next. - **Local Jacobian:** The derivative of an operation's output with respect to its inputs, evaluated at the stored forward-pass values.

Misconception alert: "The graph is just a theoretical abstraction." No. Modern frameworks (TensorFlow, PyTorch) literally build this graph in memory. When you call `.backward()` on a tensor, the framework traverses this graph. Understanding the graph means understanding what your framework does. It's not a metaphor — it's the actual data structure.

Quick check: In the expression $z = (x + y) \cdot \sigma(x)$, identify all nodes, edges, and the root node.

Answer Nodes: x, y, $[+]$, $v_1 = x + y$, $[\sigma]$, $v_2 = \sigma(x)$, $[\times]$, z. Root: z. Leaf: x, y. Edges: $x \to [+]$, $y \to [+]$, $x \to [\sigma]$, $v_1 \to [\times]$, $v_2 \to [\times]$.

---

3. The Chain Rule: From Single Paths to Multiple Inputs

3.1 Single Path: The Basic Chain Rule

For a single path $u^{(1)} \to u^{(2)} \to ... \to u^{(n)}$, the derivative of the end with respect to the beginning is the product of the step-by-step derivatives:

$$\frac{\partial u^{(n)}}{\partial u^{(1)}} = \frac{\partial u^{(n)}}{\partial u^{(n-1)}} \cdot \frac{\partial u^{(n-1)}}{\partial u^{(n-2)}} \cdot ... \cdot \frac{\partial u^{(2)}}{\partial u^{(1)}}$$

Intuition: Each factor represents a "sensitivity": how much does $u^{(k)}$ change when $u^{(k-1)}$ changes by a tiny amount? The product chains these sensitivities: a small change in $u^{(1)}$ propagates through each step to produce a change in $u^{(n)}$. This is like a cascade of gears: each gear ratio multiplies the effect of turning the first gear, and the total effect is the product of all ratios.

3.2 Multiple Paths: The Multivariate Chain Rule

When a node $u^{(i)}$ influences multiple downstream nodes $u^{(j_1)}, u^{(j_2)}, ...$, the total gradient sums over all paths:

$$\frac{\partial J}{\partial u^{(i)}} = \sum_{j:u^{(i)} \text{ is input to } u^{(j)}} \frac{\partial J}{\partial u^{(j)}} \cdot \frac{\partial u^{(j)}}{\partial u^{(i)}}$$

This is the single most important equation in backpropagation. Let's parse it:

- $\frac{\partial J}{\partial u^{(j)}}$ is the **upstream gradient**: the gradient of the final cost with respect to the output of operation j. We already computed this because we are working backwards.
- $\frac{\partial u^{(j)}}{\partial u^{(i)}}$ is the **local Jacobian**: how much does this operation's output change when its i-th input changes? We compute this using the values stored during the forward pass.
- The product is the contribution of this one path. The sum collects contributions from all paths where $u^{(i)}$ feeds into something.

Worked example: Let $J = u^{(3)} = u^{(1)} \cdot u^{(2)}$, with $u^{(1)} = 3$ and $u^{(2)} = 4$. Then $u^{(3)} = 12$. The operation is multiplication.

- $\frac{\partial J}{\partial u^{(3)}} = 1$ (the gradient of the output with respect to itself).
- $\frac{\partial u^{(3)}}{\partial u^{(1)}} = u^{(2)} = 4$ (local Jacobian for first input).
- $\frac{\partial u^{(3)}}{\partial u^{(2)}} = u^{(1)} = 3$ (local Jacobian for second input).

Applying the multivariate chain rule:

$$\frac{\partial J}{\partial u^{(1)}} = \frac{\partial J}{\partial u^{(3)}} \cdot \frac{\partial u^{(3)}}{\partial u^{(1)}} = 1 \cdot 4 = 4$$

$$\frac{\partial J}{\partial u^{(2)}} = \frac{\partial J}{\partial u^{(3)}} \cdot \frac{\partial u^{(3)}}{\partial u^{(2)}} = 1 \cdot 3 = 3$$

Notice: The local Jacobian for multiplication uses the *other* input's value. For $u^{(1)}$, we needed $u^{(2)}$. This is why we stored all intermediate values during the forward pass. It's not a design choice — it's a mathematical necessity. The gradient of multiplication with respect to one input *is* the other input. Without storing that value, backpropagation would be blind.

Try this yourself: What happens if you don't store $u^{(2)}$ during the forward pass? Can you still compute $\frac{\partial J}{\partial u^{(1)}}$ exactly?

Answer No. The local Jacobian $\frac{\partial u^{(3)}}{\partial u^{(1)}} = u^{(2)}$ requires the *runtime value* of $u^{(2)}$, not its symbolic form. If $u^{(2)}$ was itself computed from earlier inputs (e.g., $u^{(2)} = \tanh(w)$), you cannot reconstruct its value from the downstream gradient alone. This is why frameworks store the entire forward activations in memory — and why training large models demands so much GPU RAM.

Misconception alert: "The chain rule is just multiplying derivatives." For scalar-valued functions with multiple inputs, it is a *sum* over incoming paths, each being a *product*. Students often forget the sum

when a variable feeds into multiple operations (e.g., skip connections in ResNets). The sum is not optional — it's the mathematical consequence of a variable influencing the output through multiple independent channels.

Quick check: Suppose $a = 2$, $b = a^2$, $c = a + b$, and $J = c$. Compute $\frac{\partial J}{\partial a}$.

Answer a influences both b and c. Path 1: $a \to c$ directly, $\frac{\partial c}{\partial a} = 1$. Path 2: $a \to b \to c$, $\frac{\partial c}{\partial b} \cdot \frac{\partial b}{\partial a} = 1 \cdot 2a = 4$. Sum: $\frac{\partial J}{\partial a} = 1 + 4 = 5$. Check: $c = a + a^2 = 2 + 4 = 6$, $\frac{dc}{da} = 1 + 2a = 5$. Correct.

---

4. The Backpropagation Algorithm: Two Passes

4.1 Forward Pass

Purpose: Compute the output of the network and store every intermediate value that will be needed during the backward pass.

The forward pass executes the computational graph in topological order: every node is computed only after all its inputs are available. For each operation, we:

1. Receive input values from upstream nodes.
2. Compute the output value.
3. **Store** the output value (and often the input values themselves) in memory.
4. Pass the output to downstream nodes.

What gets stored and why: For a multiplication node $z = x \cdot y$, we need both x and y during the backward pass because $\frac{\partial z}{\partial x} = y$ and $\frac{\partial z}{\partial y} = x$. For a sigmoid node $a = \sigma(z)$, we could store a and reconstruct $\sigma'(z) = a(1 - a)$ during the backward pass — but many implementations store z directly for numerical stability. The rule: store whatever the local Jacobian needs to evaluate.

The memory cost: This is the forward pass's hidden burden. Training a deep network means storing activations for every layer, for every sample in a minibatch. A ResNet-50 with a batch size of 32 stores tens of millions of floating-point numbers. This is why GPU memory is the bottleneck in deep learning, not compute speed. In fact, the forward pass is a kind of strategic hoarding: we're trading memory for time, storing values now to avoid recomputing them later. This is the same space-time tradeoff that makes dynamic programming work — and it's why techniques like gradient checkpointing (which recomputes activations on-the-fly to save memory) exist.

4.2 Backward Pass

Purpose: Compute $\frac{\partial J}{\partial \text{every leaf node}}$ by traversing the graph in reverse topological order.

The backward pass starts at the root node (the cost J) with an upstream gradient of $\frac{\partial J}{\partial J} = 1$. It then proceeds backward:

Algorithm (per operation node):

1. Receive the upstream gradient $\frac{\partial J}{\partial u^{(j)}}$ (the gradient of the cost with respect to *this* node's output).
2. Retrieve the stored input values from the forward pass.
3. Compute the local Jacobian: for each input $u^{(i)}$, compute $\frac{\partial u^{(j)}}{\partial u^{(i)}}$.
4. Multiply: $\frac{\partial J}{\partial u^{(i)}} \leftarrow \frac{\partial J}{\partial u^{(j)}} \cdot \frac{\partial u^{(j)}}{\partial u^{(i)}}$.
5. If $u^{(i)}$ has multiple outgoing edges, *add* this contribution to any previously computed gradient for $u^{(i)}$.
6. Pass the result downstream (toward the inputs).

Why reverse topological order matters: When we reach a node, we must already have computed the upstream gradient — the gradient of the cost with respect to *its* output. Reverse topological order guarantees this: we process nodes from output to input, so every node's consumers have already been processed.

The dynamic programming insight: Step 5 is where the magic happens. When a node $u^{(i)}$ feeds into multiple operations, we don't recompute its gradient from scratch. We accumulate contributions as we visit each consumer. By the time we finish processing all consumers, $\frac{\partial J}{\partial u^{(i)}}$ is complete — and we computed it exactly once. This is the "reuse" that collapses exponential time to linear time.

Try this yourself: Trace the backward pass for $z = (x + y) \cdot \sigma(x)$ with $x = 1, y = 2$. Compute all gradients by hand.

Answer Forward: $v_1 = x + y = 3$, $v_2 = \sigma(1) \approx 0.731$, $z = 3 \cdot 0.731 = 2.193$.

Backward: Start with $\frac{\partial J}{\partial z} = 1$ (assuming $J = z$).

Node [×]: inputs v_1, v_2. - $\frac{\partial z}{\partial v_1} = v_2 = 0.731$, so $\frac{\partial J}{\partial v_1} = 1 \cdot 0.731 = 0.731$ - $\frac{\partial z}{\partial v_2} = v_1 = 3$, so $\frac{\partial J}{\partial v_2} = 1 \cdot 3 = 3$

Node [+]: inputs x, y. - $\frac{\partial v_1}{\partial x} = 1$, so $\frac{\partial J}{\partial x} \leftarrow 0.731 \cdot 1 = 0.731$ - $\frac{\partial v_1}{\partial y} = 1$, so $\frac{\partial J}{\partial y} = 0.731 \cdot 1 = 0.731$

Node [σ]: input x. - $\frac{\partial v_2}{\partial x} = \sigma(1)(1 - \sigma(1)) = 0.731 \cdot 0.269 \approx 0.197$ - $\frac{\partial J}{\partial x} \leftarrow 3 \cdot 0.197 = 0.591$

Now x has two contributions: 0.731 from the [+] path and 0.591 from the [σ] path. Sum: $\frac{\partial J}{\partial x} = 0.731 + 0.591 = 1.322$. Final: $\frac{\partial J}{\partial y} = 0.731$.

---

5. Backpropagation in a Two-Layer Network

Let's apply the algorithm to a concrete neural network with one hidden layer. This is where the abstract graph becomes a real training step.

Architecture: - Input: $x \in \mathbb{R}^d$ (a vector of d features) - Hidden layer: $h = \sigma(W^{[1]}x + b^{[1]})$ where $W^{[1]} \in \mathbb{R}^{h \times d}$, $b^{[1]} \in \mathbb{R}^h$ - Output: $\hat{y} = W^{[2]}h + b^{[2]}$ where $W^{[2]} \in \mathbb{R}^{k \times h}$, $b^{[2]} \in \mathbb{R}^k$ - Loss: $J = \frac{1}{2}\|\hat{y} - y\|_2^2$ (squared error for simplicity)

The computational graph (for a single input, omitting batch dimension):

`x ⟶ [W¹x+b¹] ⟶ z¹ ⟶ [ ] ⟶ h ⟶ [W²h+b²] ⟶ ŷ ⟶ [loss] ⟶ J`

Forward pass (store everything in brackets):

1. $z^{[1]} = W^{[1]}x + b^{[1]}$ — store x, $W^{[1]}$, $b^{[1]}$, $z^{[1]}$
2. $h = \sigma(z^{[1]})$ — store $z^{[1]}$ (or h, but $z^{[1]}$ is safer numerically)
3. $\hat{y} = W^{[2]}h + b^{[2]}$ — store h, $W^{[2]}$, $b^{[2]}$, $\hat{y}$
4. $J = \frac{1}{2}\|\hat{y} - y\|^2$ — store $\hat{y}$, y

Backward pass:

Start with $\frac{\partial J}{\partial J} = 1$.

Step 1: Loss node. $\frac{\partial J}{\partial \hat{y}} = \hat{y} - y$ (vector of errors, one per output unit). This is the upstream gradient for the output layer.

Step 2: $W^{[2]}h+b^{[2]}$ operation. This is a linear (affine) transformation. For a single operation $\hat{y} = Wh+b$:

- $\frac{\partial \hat{y}}{\partial h} = W^\top$ (the weight matrix transposed — think about dimensions: $\hat{y} \in \mathbb{R}^k$, $h \in \mathbb{R}^h$, so the